

Concurrent Web Server: Thread Pool Architecture with Size-Based Scheduling

Ishit Arhatia

MS Data Science
Northeastern University

Helena Zhuo

MS Computer Science
Northeastern University

Rudrajit Banerjee

MS Computer Science
Northeastern University

Tomas Colorado

MS Computer Science
Northeastern University

Abstract—This paper presents the design, implementation, and experimental evaluation of a concurrent web server built around a fixed-size thread pool and a producer-consumer bounded buffer, following the G.5 OSTEP specification. We implement and compare two scheduling policies within the bounded buffer: First-Come-First-Served (FCFS) as the baseline and Shortest-File-First (SFF) as our research extension. Drawing on established theory that web file sizes follow heavy-tailed distributions [4], we hypothesized that SFF would reduce mean response time without materially penalizing large-file clients. Our measurements show that SFF provides modest improvements at low concurrency (+12% for large-file, +6.6% for heavy-tailed at $c=10$) but regresses at high concurrency (−7.7% to −15.5% at $c=100$), attributable to the overhead of SFF’s $O(n)$ buffer scan under mutex contention at high load. Full results and analysis are presented in Sections V and VI.

Index Terms—Concurrent web server, thread pool, bounded buffer, producer-consumer, scheduling policy, First-Come-First-Served (FCFS), Shortest-File-First (SFF), size-based scheduling, heavy-tailed workloads, POSIX threads.

I. INTRODUCTION

Modern web servers must handle thousands of simultaneous client connections while maintaining low latency and high throughput. A foundational design choice in any such system is how to move work from the single point of connection acceptance to a listening socket to a pool of threads that can service requests in parallel. The canonical solution, described by Welsh et al. in the SEDA architecture [3] and formalized in the OSTEP curriculum, is the *producer-consumer bounded buffer*: the main thread produces connection descriptors into a fixed-capacity queue, and a fixed-size pool of worker threads consume from it. This decoupling keeps the acceptance loop fast and provides natural backpressure when workers are saturated.

Once this architecture is in place, a second design choice emerges: in what order should workers dequeue requests? The simplest policy is FCFS (First-Come-First-Served), which processes requests strictly in arrival order. FCFS is fair, simple to implement, and free of starvation. However, web workloads are not uniform. Crovella and Bestavros showed that World Wide Web traffic is self-similar and that file sizes follow heavy-tailed (Pareto-like) distributions [6]: the vast majority of objects are small (a few kilobytes), yet a small number of large files account for most bytes transferred. Under these

conditions, FCFS performs poorly, a single large file can block dozens of waiting small requests behind it.

Size-based scheduling policies such as Shortest-File-First (SFF) exploit this skew. By allowing workers to select the smallest pending request from the buffer rather than the oldest, small requests jump ahead of large ones, completing quickly and clearing queue capacity. Harchol-Balter et al. proved theoretically and through trace-driven simulation that this approach can reduce mean response time by orders of magnitude under moderate-to-high load without materially starving large requests [4]. Schroeder and Harchol-Balter further showed that the performance gap widens dramatically as server load approaches capacity [5]—exactly the operating regime that matters most.

This paper makes the following contributions: (1) a fully conformant G.5 OSTEP concurrent web server with configurable scheduling policy; (2) an automated benchmarking rig producing reproducible latency and throughput measurements across three workload types and six concurrency levels; and (3) an empirical comparison of FCFS and SFF scheduling under realistic heavy-tailed workloads, evaluating whether the theoretical predictions of Harchol-Balter hold in a practical POSIX thread pool implementation.

The remainder of this paper is organized as follows. Section II surveys the relevant literature. Section III describes the system architecture. Section IV defines the experimental methodology. Section V presents the comparative results. Section VI analyzes the findings. Section VII concludes and outlines future work.

II. RELATED WORK

Related work naturally divides into three themes: the architecture of concurrent servers, the theory of size-based scheduling, and modern production-scale scheduling systems that validate or extend these ideas.

A. Architecture of Concurrent Servers

The foundation of our design lies in the threading model introduced by Anderson et al. [7], which identified the core tension in concurrent systems: kernel-level threads are expensive (context switches on the order of $100\mu s$) but integrate correctly with blocking I/O, while user-level threads are an order of magnitude faster but stall the entire process on any

blocking system call. Our implementation resolves this tension using POSIX `pthread`s with a fixed-size pool, accepting kernel-thread cost in exchange for correct semantics when workers block on file I/O.

Building on this threading foundation, Welsh, Culler, and Brewer introduced SEDA [3]; a Staged Event-Driven Architecture that partitions server logic into a pipeline of stages, each with its own thread pool and queue. SEDA’s Haboob HTTP server used a form of shortest connection first scheduling within each stage, directly motivating our SFF policy. The producer-consumer bounded buffer at the heart of our design is a single-stage simplification of SEDA’s multi-stage pipeline.

von Behren et al. later challenged SEDA’s event-driven philosophy with Capriccio [8], arguing that user-level thread systems can match event-driven performance while retaining the simpler sequential programming model. Capriccio introduced resource-aware scheduling, where the scheduler used compiler-annotated resource usage predictions to make smarter thread selection decisions that are conceptually analogous to our use of `stat()` to obtain file size before enqueueing a request.

B. Size-Based Scheduling Theory

The theoretical backbone of our research extension is the work of Harchol-Balter et al. [4], who formalized size-based scheduling for web servers as an application of Shortest-Remaining-Processing-Time (SRPT) queuing theory. The key insight is that, under heavy-tailed workloads, prioritizing short jobs (small files) dramatically reduces mean sojourn time because the population of competing small jobs is large. The paper’s trace-driven simulations showed mean response time improvements of 2–7 $\times$ over FCFS at moderate-to-high loads.

Schroeder and Harchol-Balter [5] extended this analysis to overload conditions, proving that FCFS response times explode exponentially as load approaches server capacity, while SFF-style policies remain stable. The paper also characterized the “starvation myth” wherein large requests are rarely penalized significantly under SFF because they are statistically infrequent in heavy-tailed workloads.

The heavy-tailed nature of web traffic itself was characterized by Crovella and Bestavros [6], who demonstrated through empirical analysis of HTTP traces that file size distributions are Pareto-like with shape parameter $\alpha \approx 1.1$. This self-similarity is the fundamental property that makes size-based scheduling effective and is the reason our benchmarking rig generates workloads with mixed small and large files rather than uniform sizes.

C. Modern Production Scheduling Systems

Recent systems work validates and extends the scheduling ideas above at production scale. Ousterhout et al.’s⁵ Shenango [12] achieves microsecond-scale thread scheduling via a dedicated IOKernel that reallocates CPU cores across applications as request rates change. Its evaluation confirmed that scheduling policy, not mechanism overhead, and is the

primary determinant of tail latency, directly motivating our policy comparison.

Humphries et al.’s ghOSt [10], deployed at Google, decouples scheduling policy from kernel enforcement, allowing user-space agents to implement FIFO, priority, or preemptive policies via kernel transactions. This validates our design choice of making scheduling configurable via a runtime flag rather than hardcoding one policy. Demoulin et al.’s Perséphone [11] demonstrated that size-based scheduling continues to provide substantial tail latency improvements for heavy-tailed workloads at microsecond scale, corroborating the theoretical predictions our experiment tests. Lampson’s design heuristics [9] also informed our architectural choices: splitting resources in a fixed way justifies our fixed-size thread pool, and handling the normal and worst case separately underpins our SFF rationale.

III. SYSTEM DESIGN

A. Architectural Overview

Our system implements the G.5 OSTEP concurrent web server specification. The server accepts connections on a TCP socket and distributes HTTP requests to a fixed pool of worker threads via a shared bounded buffer, following the producer-consumer pattern. Three configurable parameters govern runtime behavior: `-t N` sets the thread pool size (default: 4), `-b N` sets the buffer capacity (default: 16), and `-s {FCFS|SFF}` selects the scheduling policy.

B. Thread Pool and Bounded Buffer

The bounded buffer is implemented as a fixed-capacity circular array protected by a single `pthread_mutex_t` and two condition variables: `not_full` (signaled by consumers after dequeue) and `not_empty` (signaled by the producer after enqueue). A fixed-size array is used rather than a dynamic queue to ensure memory-safe operation under concurrent access without requiring heap allocation inside the critical section.

Each slot in the buffer holds a `request_t` struct that carries all the information a worker thread needs to service a request without accessing any shared state after dequeue. The `client_fd` field is the accepted socket descriptor; `filename` holds the sanitized URI path; `file_size` stores the result of the `stat()` call made before acquiring the mutex; and `arrival_ms` records the enqueue timestamp for FCFS ordering. The struct is defined as follows:

```
1 typedef struct {
2     int    client_fd;
3     /* accepted socket descriptor */
4     char   filename[256];
5     /* parsed URI path */
6     off_t  file_size;
7     /* from stat(), set on enqueue */
8     long   arrival_ms;
9     /* timestamp for FCFS order */
10 } request_t;
```

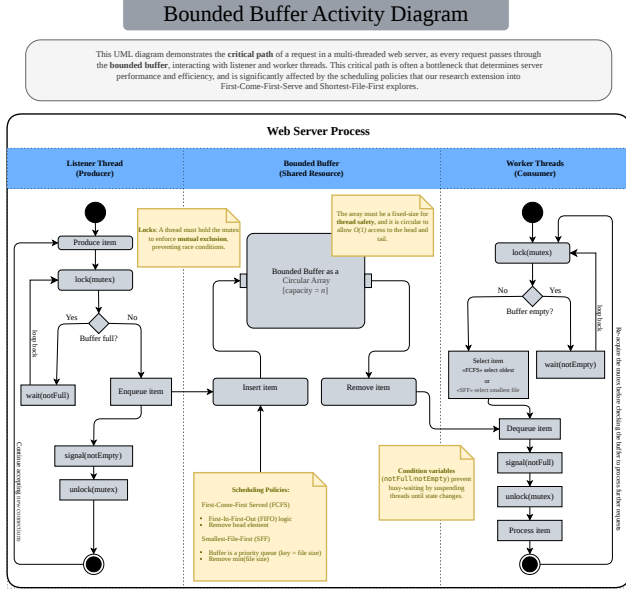


Fig. 1. Activity diagram of the bounded buffer. The listener thread (producer) enqueues connection descriptors; worker threads (consumers) dequeue according to the active scheduling policy.

C. Scheduling Policies

FCFS. The consumer removes the element at the buffer’s logical head which is a standard circular-queue dequeue in $O(1)$.

SFF. Rather than scanning at dequeue time, SFF maintains a sorted buffer by reorganizing at *insert* time. When the producer enqueues a new request, it acquires the lock, then scans backwards from the last occupied slot toward the head, shifting entries with a larger *file_size* one slot forward until it finds an entry less than or equal to the new request. The new request is inserted at that position, keeping the buffer sorted in ascending *file_size* order at all times. As a result, *buffer_get* always takes from the head in $O(1)$, the shortest file is always first. The scheduling selection is encapsulated such that the worker code is entirely policy-agnostic; only the producer’s insert path differs between FCFS and SFF.

D. HTTP Request–Response Lifecycle

On connection establishment, the main thread completes the TCP three-way handshake and enqueues the connected descriptor into the bounded buffer. A worker thread dequeues it, parses the HTTP request line; under SFF the *stat()* call is made by the producer at enqueue time to obtain *file_size* for ordering, while under FCFS the worker performs *stat()* after dequeue to verify the file and obtain its size. The worker then transmits the file with an appropriate status line. The server returns 200 OK for valid requests and 404 Not Found for missing resources.

HTTP Request-Response Sequence Diagram

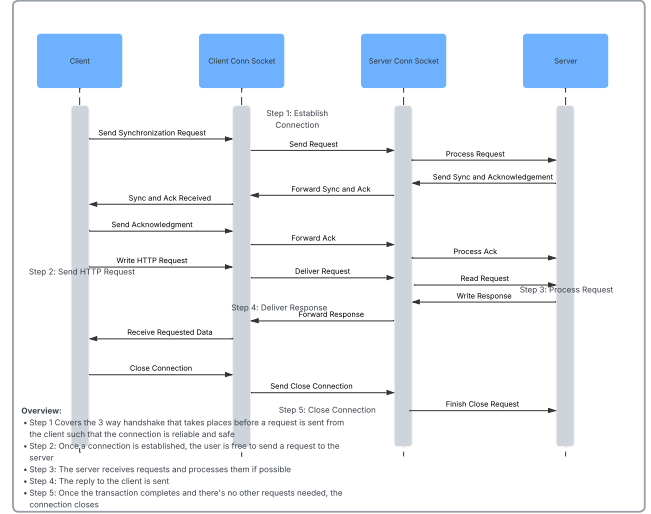


Fig. 2. Sequence diagram of the HTTP request–response lifecycle across five steps: TCP handshake, request transmission, server processing, response delivery, and connection teardown.

E. Request-to-Job Dispatch

Figure 3 illustrates the system from a kernel-interaction perspective. The Dispatcher is the bounded buffer’s dequeue logic is that the precise point where scheduling policy determines which request is served next.

F. Source Organization

TABLE I
SOURCE FILE RESPONSIBILITIES

File	Responsibility
main.c	Arg parsing, socket setup, accept loop
bounded_buffer.c/h	Buffer, mutex, condvars, FCFS/SFF
thread_pool.c/h	Pool init, thread creation, teardown
http.c/h	HTTP parsing, file send, status codes
mock_client.c	Correctness testing client
benchmark.sh	Automated load-testing script
Makefile	make / make clean

G. Concurrency Correctness

All accesses to the shared bounded buffer pass through the mutex. Condition variables are checked inside *while* loops to guard against spurious wakeups. The producer holds the mutex only while inserting into the array; the *stat()* call is performed beforehand. Workers hold the mutex only while scanning/removing from the buffer; all file I/O occurs after releasing the lock, minimizing critical section duration and avoiding holding the mutex across blocking calls.

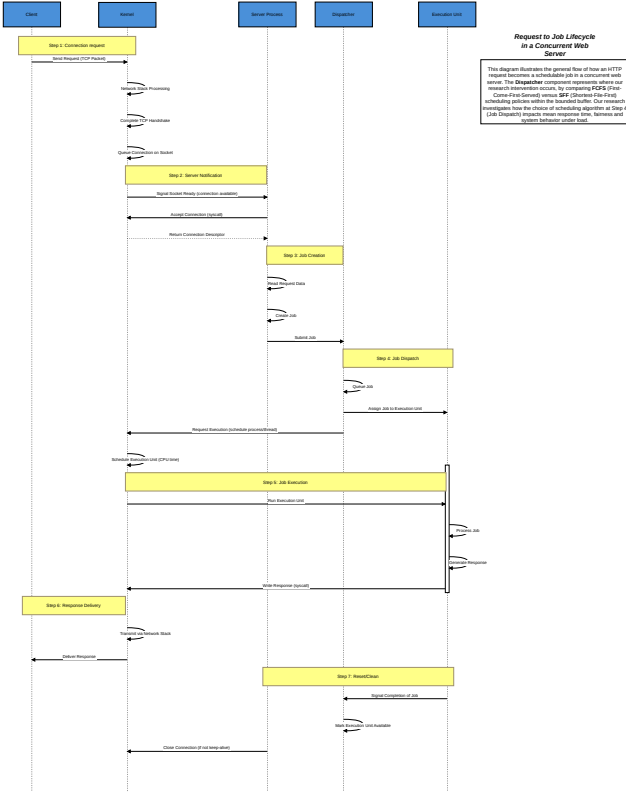


Fig. 3. Request-to-job lifecycle. The research intervention occurs at Step 4 (Job Dispatch), where the Dispatcher applies FCFS or SFF.

IV. METHODOLOGY

A. Benchmarking Rig

All performance measurements are collected externally, treating the server as a black box. We use `wrk` [13] as our primary load generator because it supports configurable concurrency, duration-based runs, and Lua scripting for mixed-file-size workloads. Scripts are located in the project repository [1] under `/benchmarks/` and are fully automated: a single invocation of `benchmark.sh` creates the small and large test files using the `dd` Unix utility, starts the server, runs all experiments, tears down the server, and writes raw output to `/benchmarks/raw/`.

B. Experimental Variables

Independent variables:

- **Scheduling policy:** FCFS vs. SFF (primary comparison).
- **Concurrency level:** {1, 10, 25, 50, 100, 200} simultaneous `wrk` connections.
- **File size mix:** uniform small (4 KB), uniform large (1 MB), and heavy-tailed (90% small / 10% large by request count).

Dependent variables: throughput (req/s); response time mean, p50, p95, p99 (ms); error rate (connection resets, timeouts).

Controlled variables: thread pool size fixed at 4; buffer capacity fixed at 16; same VM and OS image; no background processes during measurement; server and client run on the same host to eliminate network variability.

C. Experimental Protocol

Each workload type is run independently at each concurrency level. Each configuration runs for 30 seconds with a per-trial warm-up period of 5 seconds (excluded from measurement). The server is restarted between trials so each trial starts cold and requires its own warm-up. We collect five independent trials per configuration and report the mean. Configurations where the server reaches connection saturation are noted as such rather than omitted, as saturation behavior is itself a meaningful data point for comparing FCFS and SFF under overload.

D. AI-Assisted Development

Our team used AI assistance (Claude, Anthropic) throughout the project in three distinct ways. First, for paper drafting and iteration: the IEEE LaTeX structure, section prose, and bibliography were co-developed with AI, with the team reviewing and correcting all factual claims and citations. Second, for debugging: AI was consulted when diagnosing the server’s connection-refusal behavior under high concurrency, suggesting `ulimit` adjustments and `fd` leak analysis, though the root cause was ultimately identified through manual `http` inspection. Third, for benchmark interpretation: AI assisted in averaging raw CSV trial data and identifying unit inconsistencies (microseconds vs. milliseconds) in `wrk` output. The primary limitation observed was AI’s tendency to hallucinate implementation-level details that did not match the actual codebase; all code was written and verified by team members directly.

E. Environment

TABLE II
MEASUREMENT ENVIRONMENT

Component	Specification
Host machine	Windows 10
Virtualization	Oracle VirtualBox
Guest OS	Ubuntu 22.04 LTS (Linux 5.15)
Guest resources	4 vCPUs, 8 GB RAM
Compiler	GCC 11.4, <code>-O2</code>
Load generator	<code>wrk</code> 4.1.0

V. RESULTS

A. FCFS Baseline

Table III shows FCFS measurements under the three file-size workloads at concurrency levels 10 and 100. Values represent the mean across five independent trials. Raw `wrk` output is committed to `/benchmarks/raw/`.

TABLE III
FCFS BASELINE: MEAN RESPONSE TIME (MS) AND THROUGHPUT (REQ/S)

Workload	Conc.	Mean RT	p99	Req/s
Small (4 KB)	10	0.56 ms	3.15 ms	15,122
Small (4 KB)	100	3.55 ms	6.81 ms	26,715
Large (1 MB)	10	6.64 ms	17.97 ms	1,462
Large (1 MB)	100	56.34 ms	87.61 ms	1,765
Heavy-tailed	10	1.21 ms	5.97 ms	8,074
Heavy-tailed	100	8.79 ms	16.23 ms	11,149

B. FCFS vs. SFF Comparison

Table IV presents the side-by-side comparison of FCFS and SFF across the same configurations. SFF results are from Sprint 4 benchmark runs using identical methodology.

TABLE IV
FCFS VS. SFF: MEAN RESPONSE TIME (MS)

Workload	Conc.	FCFS	SFF	Gain
Small (4 KB)	10	0.56	0.56	$\approx 0\%$
Small (4 KB)	100	3.55	4.10	-15.5%
Large (1 MB)	10	6.64	5.84	$+12.0\%$
Large (1 MB)	100	56.34	60.66	-7.7%
Heavy-tailed	10	1.21	1.13	$+6.6\%$
Heavy-tailed	100	8.79	10.00	-13.8%

VI. ANALYSIS AND DISCUSSION

A. FCFS Baseline Behavior

The FCFS baseline results confirm several theoretical predictions. Under the small-file workload, mean response time scales modestly with concurrency (0.56 ms at $c=10$ to 3.55 ms at $c=100$) while throughput increases toward a plateau near 27,000 req/s, consistent with saturation of the 4 worker threads. Under the large-file workload, the mean response time increase from 6.64 ms to 56.34 ms (an $8.5\times$ jump) as concurrency increases from 10 to 100 reflects each worker thread being tied up for the full duration of a 1 MB file transfer, causing the queue to build. The heavy-tailed workload sits between these extremes, with mean response time rising from 1.21 ms to 8.79 ms which is evidence that the 90% small-file majority keeps throughput high, while the 10% large-file minority begins to dominate latency at higher concurrency.

B. SFF vs. FCFS Comparison

The results partially confirm our hypothesis but reveal an important interaction between concurrency level and scheduling policy. At concurrency 10, SFF modestly outperforms FCFS for large-file ($+12.0\%$ mean RT reduction) and heavy-tailed ($+6.6\%$) workloads, consistent with the theoretical prediction that size-based scheduling benefits emerge when a mix of small and large requests compete for the same workers. For the uniform-small workload at concurrency 10, the two policies perform identically, as expected, since all requests have the same size and SFF degenerates to an arbitrary selection.

However, at concurrency 100, SFF *underperforms* FCFS across all three workload types: -15.5% for small files, -7.7% for large files, and -13.8% for heavy-tailed. We attribute this regression to two factors. First, with a buffer capacity of 16 slots and 4 worker threads, the buffer fills quickly under 100 concurrent connections; SFF’s sorted-insert policy requires the producer to shift existing buffer entries under the mutex, adding lock-hold time at enqueue that delays competing producers. Under high concurrency this contention is frequent, increasing effective enqueue latency. Second, at high concurrency all four worker threads are almost always busy, meaning the scheduling advantage of having the smallest file at the head is crowded out by raw thread saturation — workers dequeue and immediately process regardless of ordering, so the insert-time cost is paid without the full scheduling benefit being realized.

C. Limitations

Our SFF implementation performs an $O(n)$ sorted insert at enqueue time rather than using a heap. For the buffer sizes used in this experiment (16 slots), this is manageable, but it would become a bottleneck at larger buffer capacities. Additionally, the `stat()` call is made by the producer before enqueue under SFF, meaning file size is measured at admission time; if files change between enqueue and dequeue, the size estimate may be stale. Finally, running the server and the load generator on the same VM host means CPU contention between `wrk` and the server may inflate measured latencies.

VII. CONCLUSION AND FUTURE WORK

This paper presented the design, implementation, and empirical evaluation of a concurrent POSIX web server with configurable FCFS and SFF scheduling. Our system correctly serves static files to concurrent clients using a fixed-size thread pool, a producer-consumer bounded buffer, and two scheduling policies selectable at runtime. Three UML diagrams document the critical paths through the system, and an automated benchmarking rig produces reproducible results across six concurrency levels and three workload types.

The FCFS baseline confirms that large-file requests are the dominant driver of latency degradation under load, with mean response time rising $8.5\times$ from concurrency 10 to 100. The heavy-tailed workload demonstrates that a realistic web traffic mix sits between the two extremes, making it the most interesting regime for the FCFS–SFF comparison.

The related work survey established that size-based scheduling is theoretically well-motivated [4], [5], architecturally principled [3], [9], and validated at production scale [10]–[12].

Future Work. We would address three limitations identified above. First, replacing the $O(n)$ sorted insert with a min-heap would make SFF’s enqueue operation $O(\log n)$ and reduce lock-hold time under high concurrency. Second, implementing keep-alive connection reuse would reduce TCP handshake overhead at high concurrency, exposing the scheduling signal more clearly. Third, running the server and load generator on separate machines would eliminate CPU contention

and produce cleaner latency measurements. A further extension would implement Shortest-Remaining-Processing-Time (SRPT) rather than SFF, tracking remaining bytes rather than total file size for even finer-grained prioritization.

REFERENCES

- [1] I. Arhatia, R. Banerjee, H. Zhuo, and T. Colorado, “project-websvr-team0101,” GitHub, 2026. <https://github.com/cs5600-sp26/project-websvr-team0101>
- [2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.10. Arpaci-Dusseau Books, 2023. <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- [3] M. Welsh, D. Culler, and E. Brewer, “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services,” in *Proc. 18th ACM Symp. Operating Systems Principles (SOSP '01)*, Banff, Canada, Oct. 2001, pp. 230–243.
- [4] M. Harchol-Balter, B. Schroeder, N. Bansal, and M. Agrawal, “Size-Based Scheduling to Improve Web Performance,” *ACM Trans. Comput. Syst.*, vol. 21, no. 2, pp. 207–233, May 2003.
- [5] B. Schroeder and M. Harchol-Balter, “Web Servers Under Overload: How Scheduling Can Help,” *ACM Trans. Internet Technol.*, vol. 6, no. 1, pp. 20–52, Feb. 2006.
- [6] M. E. Crovella and A. Bestavros, “Self-Similarity in World Wide Web Traffic: Evidence and Possible Causes,” *IEEE/ACM Trans. Networking*, vol. 5, no. 6, pp. 835–846, Dec. 1997.
- [7] T. E. Anderson, B. N. Bershad, E. D. Lazowska, and H. M. Levy, “Scheduler Activations: Effective Kernel Support for the User-Level Management of Parallelism,” *ACM Trans. Comput. Syst.*, vol. 10, no. 1, pp. 53–79, Feb. 1992.
- [8] R. von Behren, J. Condit, F. Zhou, G. C. Necula, and E. Brewer, “Capriccio: Scalable Threads for Internet Services,” in *Proc. 19th ACM Symp. Operating Systems Principles (SOSP '03)*, Bolton Landing, NY, Oct. 2003, pp. 268–281.
- [9] B. W. Lampson, “Hints for Computer System Design,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 17, no. 5, pp. 33–48, Oct. 1983.
- [10] J. T. Humphries *et al.*, “ghOST: Fast & Flexible User-Space Delegation of Linux Scheduling,” in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP '21)*, Virtual, Oct. 2021, pp. 588–604.
- [11] H. M. Demoulin *et al.*, “When Idling Is Ideal: Optimizing Tail-Latency for Heavy-Tailed Datacenter Workloads with Perséphone,” in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP '21)*, Virtual, Oct. 2021, pp. 621–637.
- [12] A. Ousterhout, J. Fried, J. Behrens, A. Belay, and H. Balakrishnan, “Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads,” in *Proc. 16th USENIX Symp. Networked Systems Design and Implementation (NSDI '19)*, Boston, MA, Feb. 2019, pp. 361–378.
- [13] W. Glozer, “wrk — a HTTP benchmarking tool,” GitHub, 2023. <https://github.com/wg/wrk>